

Proyecto Final de Optimización Modelo Matemático Formal

Optimización de la Red Logística

Acuícola Real del Meta

Asignatura: Optimización / Investigación de Operaciones

Proyecto: Red logística acuícola con grafos, PL y metaheurísticas

Equipo: [Nombres de los integrantes]

Docente: [Nombre del docente]

Fecha: 30 de mayo de 2026

Documento de Fase 1:

Formulación matemática, explicación didáctica y validación con un caso aplicado pequeño.

Índice

1. Introducción	4
1.1. Idea general del modelo	4
2. Representación del sistema como grafo dirigido ponderado	5
2.1. Tipos de nodos	5
2.2. Tipos de rutas o aristas	6
3. Lectura intuitiva antes de formular matemáticamente	7
4. Conjuntos, índices y parámetros	8
4.1. Conjuntos e índices	8
4.2. Parámetros de oferta y demanda	8
4.3. Parámetros de costos	8
4.4. Parámetros de capacidad, calidad y merma	9
5. Variables de decisión	9
5.1. Variables continuas	10
5.2. Variables binarias e integer opcionales	10
6. Modelo matemático base de Programación Lineal	10
6.1. Función objetivo de minimización de costos	11
6.2. Explicación sencilla de la función objetivo	11
7. Restricciones del modelo	12
7.1. Restricción de oferta en estaciones de producción	12
7.2. Restricción de capacidad de rutas	12
7.3. Restricción de capacidad de acopios	12
7.4. Balance de flujo con merma en acopios	13

7.5. Cumplimiento de demanda	13
7.6. Restricción de calidad	14
7.7. Restricción de camiones	14
7.8. No negatividad	15
8. Modelo compacto completo	15
9. Relación con teoría de grafos	16
9.1. Ruta de costo mínimo	16
9.2. Flujo máximo y cuellos de botella	17
10.Estrategia híbrida: algoritmo genético y gradiente	18
10.1. Por qué no reemplazan al modelo lineal	18
10.2. Algoritmo genético para decisiones de alto impacto	18
10.3. Gradiente para refinamiento local	19
11.Análisis de sensibilidad	20
11.1. Escenario 1: aumento del combustible	20
11.2. Escenario 2: cierre de vía principal	20
11.3. Escenario 3: falla masiva de calidad	20
12.Ejemplo aplicado de validación del modelo	21
12.1. Estructura del caso	21
12.2. Datos de oferta, capacidad y merma	21
12.3. Costos unitarios de transporte	22
12.4. Modelo específico del ejemplo	23
12.5. Solución óptima obtenida	23
12.6. Verificación de restricciones	25
12.6.1. Verificación de oferta	25
12.6.2. Verificación de capacidad de acopios	25

12.6.3. Verificación de balance con merma	25
12.6.4. Verificación de demanda	26
12.6.5. Verificación de rutas saturadas	26
12.7. Interpretación logística de la solución	27
13. Mini análisis de sensibilidad sobre el caso aplicado	27
13.1. Aumento del 15 % en costos de rutas del Meta	27
13.2. Cierre de la ruta $O_1 \rightarrow A_1$	28
14. Traducción del modelo a Python	29
15. Conclusiones de la fase matemática	29
Referencias bibliográficas sugeridas	30
A. Resumen de símbolos principales	30
B. Recomendación para defensa oral	31

1. Introducción

La empresa **Acuícola Real del Meta** necesita coordinar el transporte de pescado desde estaciones de producción hasta supermercados, pasando por centros de acopio intermedios. El problema no consiste únicamente en encontrar una ruta corta. En realidad, se debe decidir **cuántas toneladas enviar por cada ruta, qué centros de acopio utilizar, cómo respetar la capacidad de camiones y almacenamiento, cómo cumplir la demanda de los supermercados y cómo reaccionar ante fallas de calidad o cierres de vías.**

Desde el punto de vista de optimización, este problema se puede interpretar como una **red de flujo**. Una red de flujo es un conjunto de nodos conectados por rutas. Los nodos representan lugares físicos: estaciones, acopios y supermercados. Las rutas representan vías de transporte. En cada ruta se puede enviar cierta cantidad de producto, pero esa cantidad tiene un límite y genera un costo.

El objetivo principal del modelo es determinar una asignación de flujos que permita:

- 1) minimizar el costo total de transporte, operación, almacenamiento y penalizaciones;
- 2) cumplir la demanda de los supermercados;
- 3) no superar la oferta de las estaciones de producción;
- 4) no superar la capacidad de rutas, camiones y centros de acopio;
- 5) incorporar deterioro o merma del producto;
- 6) considerar restricciones de calidad;
- 7) permitir análisis de escenarios como aumento del combustible, cierre de vías y fallas de calidad.

La formulación principal se presenta como un modelo de **Programación Lineal** (PL). Sin embargo, cuando se decide si una ruta está activa o no mediante variables binarias, el modelo se convierte en un problema de **Programación Lineal Entera Mixta** (PLEM o MILP, por sus siglas en inglés). Esta aclaración es importante: el núcleo matemático de flujo es lineal, pero algunas decisiones logísticas reales, como abrir o cerrar rutas, son discretas.

1.1. Idea general del modelo

La idea se puede resumir así:

El modelo recibe datos de oferta, demanda, costos, distancias, capacidades y calidad. Luego decide el flujo óptimo de toneladas por cada ruta para minimizar el costo total y garantizar que la red funcione de manera factible.

Por ejemplo, si una estación produce 80 toneladas y existen dos rutas posibles hacia dos acopios, el modelo debe decidir cuántas toneladas enviar a cada acopio. Si un acopio está más cerca pero casi lleno, tal vez no convenga enviar todo allí. Si una ruta es barata pero tiene poca capacidad, el flujo debe dividirse. Si un acopio presenta falla de calidad, el modelo debe detener el flujo por ese nodo o desviarlo aplicando una penalización.

2. Representación del sistema como grafo dirigido ponderado

El sistema logístico se representa como un **grafo dirigido ponderado**:

$$G = (V, E) \tag{1}$$

Donde:

- V es el conjunto de vértices o nodos.
- E es el conjunto de aristas o rutas dirigidas.
- Cada arista $(i, j) \in E$ representa una ruta que permite transportar producto desde el nodo i hasta el nodo j .
- Cada arista tiene pesos asociados: costo, distancia, tiempo, capacidad, riesgo y estado operativo.

En este proyecto, el grafo es **dirigido** porque no necesariamente transportar de i hacia j cuesta lo mismo que transportar de j hacia i , e incluso puede ocurrir que la ruta inversa no exista dentro de la operación logística.

2.1. Tipos de nodos

El conjunto total de nodos se divide en tres subconjuntos:

$$V = O \cup A \cup D \quad (2)$$

$$O \cap A = \emptyset, \quad O \cap D = \emptyset, \quad A \cap D = \emptyset \quad (3)$$

Donde:

- O : conjunto de **orígenes** o estaciones de producción.
- A : conjunto de **centros de acopio** o nodos intermedios.
- D : conjunto de **destinos** o supermercados.

Para el caso completo del proyecto se espera una red de tamaño aproximado:

$$O = \{o_1, o_2, o_3, o_4, o_5, o_6\}, \quad (4)$$

$$A = \{a_1, a_2, \dots, a_{10}\}, \quad (5)$$

$$D = \{d_1, d_2, \dots, d_k\}, \quad 25 \leq k \leq 30. \quad (6)$$

Por tanto, el número total de nodos está aproximadamente entre:

$$|V| = |O| + |A| + |D| = 6 + 10 + k, \quad 25 \leq k \leq 30 \quad (7)$$

$$41 \leq |V| \leq 46. \quad (8)$$

2.2. Tipos de rutas o aristas

El conjunto de rutas se divide en dos grupos principales:

$$E = E^{OA} \cup E^{AD} \quad (9)$$

Donde:

- E^{OA} : rutas desde orígenes hacia acopios.

- E^{AD} : rutas desde acopios hacia destinos.

En una versión inicial del proyecto se recomienda trabajar con una estructura de dos niveles:

$$\text{Origen} \longrightarrow \text{Acopio} \longrightarrow \text{Supermercado} \quad (10)$$

Una versión más avanzada podría permitir rutas directas origen-supermercado o rutas entre acopios. Sin embargo, para la primera fase es más conveniente mantener la estructura jerárquica porque facilita la formulación, la depuración del código y la explicación ante el profesor.

3. Lectura intuitiva antes de formular matemáticamente

Antes de escribir ecuaciones, conviene entender qué se está decidiendo.

Suponga que una estación o_1 tiene 100 toneladas de pescado disponibles. Existen dos centros de acopio: a_1 y a_2 . El modelo debe decidir:

$$x_{o_1 a_1} = \text{toneladas enviadas desde } o_1 \text{ hasta } a_1 \quad (11)$$

$$x_{o_1 a_2} = \text{toneladas enviadas desde } o_1 \text{ hasta } a_2 \quad (12)$$

Si $x_{o_1 a_1} = 60$ y $x_{o_1 a_2} = 40$, entonces se envían 60 toneladas al primer acopio y 40 al segundo. La suma enviada por o_1 es:

$$x_{o_1 a_1} + x_{o_1 a_2} = 60 + 40 = 100 \quad (13)$$

Esto no puede superar la oferta de o_1 . Si o_1 solo tiene 100 toneladas, no sería posible enviar 130.

Del mismo modo, si el supermercado d_1 pide 45 toneladas, la suma que llega desde todos los acopios debe ser 45:

$$x_{a_1 d_1} + x_{a_2 d_1} = 45 \quad (14)$$

Eso es una restricción de demanda. La optimización aparece porque existen muchas formas posibles de cumplir esas cantidades, pero unas son más baratas, más seguras o más eficientes que otras.

4. Conjuntos, índices y parámetros

4.1. Conjuntos e índices

Símbolo	Descripción
O	Conjunto de estaciones de producción u orígenes. Índice usual: i .
A	Conjunto de centros de acopio. Índice usual: j .
D	Conjunto de supermercados o destinos. Índice usual: k .
V	Conjunto total de nodos, donde $V = O \cup A \cup D$.
E	Conjunto total de rutas o aristas dirigidas.
E^{OA}	Rutas desde estaciones de producción hacia acopios.
E^{AD}	Rutas desde acopios hacia supermercados.
T	Conjunto de periodos de tiempo si el modelo se extiende a varios días. En la primera versión puede tomarse un solo periodo.

En la versión de un solo periodo, no es necesario usar índice de tiempo. En una versión diaria o semanal se puede agregar $t \in T$ a las variables, por ejemplo x_{ij}^t .

4.2. Parámetros de oferta y demanda

Parámetro	Descripción
b_i	Oferta disponible en el origen $i \in O$, medida en toneladas.
d_k	Demanda requerida por el supermercado $k \in D$, medida en toneladas.
p_k	Precio de venta por tonelada en el destino $k \in D$. Si se usa un precio único, se puede escribir simplemente p .

4.3. Parámetros de costos

Parámetro	Descripción
-----------	-------------

c_{ij}	Costo unitario de transportar una tonelada desde i hasta j , para $(i, j) \in E$. Puede construirse como distancia por costo por tonelada-kilómetro.
g_j	Costo de operación o manipulación por tonelada en el acopio j .
h_j	Costo de almacenamiento por tonelada en el acopio j .
π_k	Penalización por cada tonelada no entregada al supermercado k .
ϕ_j	Penalización económica asociada a una falla de calidad en el acopio j .
F_{ij}	Costo fijo por activar la ruta (i, j) . Este parámetro solo se usa si se modela apertura/cierre de rutas con variables binarias.

4.4. Parámetros de capacidad, calidad y merma

Parámetro	Descripción
u_{ij}	Capacidad máxima de transporte de la ruta (i, j) en toneladas.
H_j	Capacidad máxima de almacenamiento o procesamiento del acopio j .
Q	Capacidad promedio de un camión, en toneladas por viaje.
ρ_j	Tasa de merma o deterioro en el acopio j . Por ejemplo, $\rho_j = 0.02$ significa pérdida del 2 %.
$\bar{q}_j$	Parámetro de calidad del acopio j . Toma valor 1 si el acopio está operativo y 0 si falla la evaluación de calidad.
δ_{ij}	Distancia física o estimada entre los nodos i y j .
τ_{ij}	Tiempo estimado de transporte por la ruta (i, j) .
α	Factor de incremento de combustible en escenarios de sensibilidad. Por ejemplo, $\alpha = 1.15$ representa aumento del 15 %.

5. Variables de decisión

Las variables de decisión son las cantidades que el modelo debe calcular. En otras palabras, son las respuestas que se esperan obtener del proceso de optimización.

5.1. Variables continuas

Variable	Interpretación
$x_{ij} \geq 0$	Toneladas transportadas desde el nodo i hasta el nodo j , para cada ruta $(i, j) \in E$. Es la variable principal del modelo.
$s_j \geq 0$	Stock o inventario final que queda en el acopio j .
$m_j \geq 0$	Toneladas perdidas por merma en el acopio j .
$w_k \geq 0$	Toneladas de demanda incumplida en el supermercado k . Se usa para evitar que el modelo se vuelva imposible si no hay suficiente oferta.
$e_k \geq 0$	Toneladas entregadas en exceso al supermercado k . Normalmente se restringe a cero si la demanda debe cumplirse exactamente.

La variable más importante es x_{ij} . Si $x_{ij} = 0$, no se envía producto por esa ruta. Si $x_{ij} = 25$, se envían 25 toneladas por la ruta (i, j) .

5.2. Variables binarias e integer opcionales

Variable	Interpretación
$y_{ij} \in \{0, 1\}$	Vale 1 si la ruta (i, j) está activa y 0 si está cerrada o no disponible.
$r_j \in \{0, 1\}$	Vale 1 si el acopio j se utiliza en la solución y 0 si no se utiliza.
$n_{ij} \in \mathbb{Z}_{\geq 0}$	Número de camiones o viajes asignados a la ruta (i, j) .

Si se incluyen y_{ij} , r_j o n_{ij} , el problema deja de ser puramente lineal continuo y pasa a ser entero mixto. Para una primera versión en Python se recomienda resolver primero el modelo continuo con rutas ya definidas; después se agrega la parte binaria mediante algoritmo genético o mediante un solucionador MILP.

6. Modelo matemático base de Programación Lineal

Esta sección presenta el modelo base. El modelo busca minimizar costos. Si los precios de venta son fijos y la demanda se cumple completamente, maximizar la ganancia es equivalente a

minimizar el costo, porque los ingresos se vuelven constantes.

6.1. Función objetivo de minimización de costos

La función objetivo suma todos los costos relevantes:

$$\text{mín } Z = C_{\text{transporte}} + C_{\text{operacion}} + C_{\text{almacenamiento}} + C_{\text{merma}} + C_{\text{incumplimiento}} + C_{\text{calidad}}. \quad (15)$$

De forma expandida:

$$\text{mín } Z = \sum_{(i,j) \in E} c_{ij}x_{ij} + \sum_{j \in A} g_j \sum_{i \in O} x_{ij} + \sum_{j \in A} h_j s_j \quad (16)$$

$$+ \sum_{j \in A} p^{\text{ref}} m_j + \sum_{k \in D} \pi_k w_k + \sum_{j \in A} \phi_j (1 - \bar{q}_j). \quad (17)$$

Donde p^{ref} representa un valor de referencia por tonelada perdida. Si la empresa desea medir ganancia, también puede definirse:

$$\text{máx } U = \sum_{k \in D} p_k (d_k - w_k) - Z. \quad (18)$$

6.2. Explicación sencilla de la función objetivo

Cada término tiene una función concreta:

- $\sum c_{ij}x_{ij}$: costo de transportar toneladas por las rutas.
- $\sum g_j \sum x_{ij}$: costo de operar o manipular producto en acopios.
- $\sum h_j s_j$: costo de dejar producto almacenado.
- $\sum p^{\text{ref}} m_j$: costo económico de la merma.
- $\sum \pi_k w_k$: castigo por no cumplir demanda.
- $\sum \phi_j (1 - \bar{q}_j)$: penalización si un acopio falla en calidad.

El modelo intenta encontrar una combinación de flujos x_{ij} que haga que esta suma sea lo más pequeña posible, sin violar las restricciones.

7. Restricciones del modelo

Las restricciones son las reglas que ninguna solución puede romper. Una solución puede ser barata, pero si incumple demanda, excede capacidad o usa un acopio dañado, no sirve.

7.1. Restricción de oferta en estaciones de producción

Cada origen solo puede enviar como máximo lo que tiene disponible:

$$\sum_{j \in A: (i,j) \in E^{OA}} x_{ij} \leq b_i, \quad \forall i \in O. \quad (19)$$

Interpretación: si una estación produce 90 toneladas, el total enviado desde esa estación no puede ser 100. Esta restricción evita que el modelo invente producto.

7.2. Restricción de capacidad de rutas

Cada ruta tiene una capacidad máxima:

$$x_{ij} \leq u_{ij}, \quad \forall (i,j) \in E. \quad (20)$$

Si se usa una variable binaria para activar o desactivar rutas:

$$x_{ij} \leq u_{ij}y_{ij}, \quad \forall (i,j) \in E. \quad (21)$$

Interpretación: si $y_{ij} = 0$, entonces la restricción queda $x_{ij} \leq 0$, por lo tanto no se puede enviar nada por esa ruta. Si $y_{ij} = 1$, la ruta queda disponible hasta su capacidad u_{ij} .

7.3. Restricción de capacidad de acopios

El total recibido por un acopio no puede superar su capacidad de almacenamiento o procesamiento:

$$\sum_{i \in O: (i,j) \in E^{OA}} x_{ij} \leq H_j, \quad \forall j \in A. \quad (22)$$

Interpretación: si Villavicencio puede procesar 120 toneladas, no se deben enviar 150 toneladas hacia ese acopio en el periodo analizado.

7.4. Balance de flujo con merma en acopios

En un acopio ocurre una transformación: entra producto, puede perderse una parte por merma, puede salir hacia supermercados y puede quedar una cantidad en stock.

Para cada acopio $j \in A$:

$$\sum_{k \in D: (j,k) \in E^{AD}} x_{jk} + s_j = (1 - \rho_j) \sum_{i \in O: (i,j) \in E^{OA}} x_{ij}. \quad (23)$$

También puede definirse explícitamente la merma:

$$m_j = \rho_j \sum_{i \in O: (i,j) \in E^{OA}} x_{ij}, \quad \forall j \in A. \quad (24)$$

Y el balance equivalente sería:

$$\sum_{i \in O} x_{ij} = \sum_{k \in D} x_{jk} + s_j + m_j, \quad \forall j \in A. \quad (25)$$

Interpretación: no todo lo que entra al acopio necesariamente llega a los supermercados. Si la merma es del 2 %, de cada 100 toneladas que entran solo 98 quedan disponibles para despacho.

7.5. Cumplimiento de demanda

Si la demanda debe cumplirse exactamente:

$$\sum_{j \in A: (j,k) \in E^{AD}} x_{jk} = d_k, \quad \forall k \in D. \quad (26)$$

Si se permite incumplimiento con penalización:

$$\sum_{j \in A: (j,k) \in E^{AD}} x_{jk} + w_k = d_k, \quad \forall k \in D. \quad (27)$$

Interpretación: si un supermercado pide 45 toneladas, debe recibir 45. Si el modelo no puede cumplir por falta de oferta o cierre de rutas, aparece w_k como cantidad incumplida, pero esa cantidad se castiga en la función objetivo.

7.6. Restricción de calidad

La calidad puede manejarse de dos formas.

Opción A: detener el flujo si falla calidad. Si $\bar{q}_j = 0$, no se permite salida desde el acopio j :

$$x_{jk} \leq u_{jk} \bar{q}_j, \quad \forall j \in A, \forall k \in D : (j, k) \in E^{AD}. \quad (28)$$

Si $\bar{q}_j = 0$, entonces $x_{jk} \leq 0$, por lo tanto el flujo desde ese acopio queda detenido.

Opción B: permitir flujo con penalización. Si la empresa permite desviar, reprocesar o inspeccionar nuevamente, se puede dejar pasar flujo pero agregar un costo alto de penalización:

$$Z = Z + \sum_{j \in A} \phi_j (1 - \bar{q}_j). \quad (29)$$

Para el proyecto se recomienda implementar ambas opciones en el software como escenarios: *bloqueo total* y *flujo penalizado*.

7.7. Restricción de camiones

Si cada camión tiene capacidad Q , el flujo en una ruta debe estar respaldado por un número entero de viajes:

$$x_{ij} \leq Q n_{ij}, \quad \forall (i, j) \in E. \quad (30)$$

$$n_{ij} \in \mathbb{Z}_{\geq 0}. \quad (31)$$

Esta restricción es opcional en la primera versión. Puede agregarse después, cuando el modelo

base ya funcione.

7.8. No negatividad

No tiene sentido enviar toneladas negativas ni tener demanda incumplida negativa:

$$x_{ij} \geq 0, \quad \forall (i, j) \in E, \quad (32)$$

$$s_j \geq 0, \quad \forall j \in A, \quad (33)$$

$$m_j \geq 0, \quad \forall j \in A, \quad (34)$$

$$w_k \geq 0, \quad \forall k \in D. \quad (35)$$

8. Modelo compacto completo

El modelo base puede resumirse así:

$$\min Z = \sum_{(i,j) \in E} c_{ij} x_{ij} + \sum_{j \in A} g_j \sum_{i \in O} x_{ij} + \sum_{j \in A} h_j s_j + \sum_{j \in A} p^{\text{ref}} m_j + \sum_{k \in D} \pi_k w_k + \sum_{j \in A} \phi_j (1 - \bar{q}_j) \quad (36)$$

sujeto a:

$$\sum_{j \in A} x_{ij} \leq b_i, \quad \forall i \in O, \quad (37)$$

$$\sum_{i \in O} x_{ij} \leq H_j, \quad \forall j \in A, \quad (38)$$

$$x_{ij} \leq u_{ij}, \quad \forall (i, j) \in E, \quad (39)$$

$$\sum_{k \in D} x_{jk} + s_j = (1 - \rho_j) \sum_{i \in O} x_{ij}, \quad \forall j \in A, \quad (40)$$

$$m_j = \rho_j \sum_{i \in O} x_{ij}, \quad \forall j \in A, \quad (41)$$

$$\sum_{j \in A} x_{jk} + w_k = d_k, \quad \forall k \in D, \quad (42)$$

$$x_{jk} \leq u_{jk} \bar{q}_j, \quad \forall j \in A, \forall k \in D, \quad (43)$$

$$x_{ij}, s_j, m_j, w_k \geq 0. \quad (44)$$

Si se agrega activación de rutas:

$$x_{ij} \leq u_{ij}y_{ij}, \quad \forall (i, j) \in E, \quad (45)$$

$$y_{ij} \in \{0, 1\}, \quad \forall (i, j) \in E. \quad (46)$$

9. Relación con teoría de grafos

El modelo de programación lineal decide cantidades. La teoría de grafos ayuda a interpretar la red, encontrar rutas óptimas individuales y detectar cuellos de botella.

9.1. Ruta de costo mínimo

Para encontrar la ruta más barata entre dos nodos se puede usar Dijkstra si todos los pesos son no negativos. El peso de una arista puede definirse como:

$$\omega_{ij} = c_{ij} + \lambda_1 \tau_{ij} + \lambda_2 (1 - \bar{q}_j) + \lambda_3 \frac{x_{ij}}{u_{ij}}, \quad (47)$$

Donde:

- c_{ij} es el costo de transporte;
- τ_{ij} es el tiempo de recorrido;
- $1 - \bar{q}_j$ penaliza fallas de calidad;
- x_{ij}/u_{ij} mide qué tan saturada está la ruta.

Para una primera versión, se puede usar simplemente:

$$\omega_{ij} = c_{ij}. \quad (48)$$

Listing 1: Pseudocódigo simplificado de Dijkstra

```
Entrada: grafo G=(V,E), nodo origen s, pesos positivos w(i,j)
Para cada nodo v en V:
    distancia[v] = infinito
```

```

    predecesor[v] = nulo
distancia[s] = 0
Q = conjunto de nodos no visitados
Mientras Q no esté vacío:
    u = nodo de Q con menor distancia[u]
    quitar u de Q
    Para cada vecino v de u:
        alternativa = distancia[u] + w(u,v)
        Si alternativa < distancia[v]:
            distancia[v] = alternativa
            predecesor[v] = u
Salida: distancias mínimas y rutas reconstruidas

```

9.2. Flujo máximo y cuellos de botella

Para analizar cuánta carga máxima puede atravesar la red se puede crear un superorigen S conectado a todas las estaciones y un superdestino T conectado desde todos los supermercados. Luego se aplica un algoritmo de flujo máximo.

Un cuello de botella aparece cuando una ruta trabaja muy cerca de su capacidad:

$$\frac{x_{ij}}{u_{ij}} \geq \theta, \quad (49)$$

Donde θ puede ser 0.90 o 0.95. Si $\theta = 0.95$, una ruta se considera crítica cuando usa al menos el 95 % de su capacidad.

Listing 2: Detección de cuellos de botella

```

Entrada: solución óptima x_ij, capacidades u_ij, umbral theta
Para cada ruta (i,j) en E:
    uso = x_ij / u_ij
    Si uso >= theta:
        marcar ruta (i,j) como cuello de botella
Ordenar rutas críticas de mayor a menor uso
Salida: lista de rutas y nodos críticos

```

10. Estrategia híbrida: algoritmo genético y gradiente

El modelo matemático anterior es el centro del proyecto. Sin embargo, para redes grandes con muchas rutas posibles, se puede combinar con dos estrategias adicionales: **algoritmo genético** y **método de gradiente**.

10.1. Por qué no reemplazan al modelo lineal

El algoritmo genético y el gradiente no deben reemplazar la formulación matemática. La formulación matemática define qué significa una solución válida. Los algoritmos son formas de buscar o mejorar soluciones.

La estrategia recomendada es:

- 1) Formular el modelo lineal base.
- 2) Resolver el modelo con todas las rutas disponibles para obtener una solución de referencia.
- 3) Identificar rutas de alto impacto: rutas costosas, saturadas, críticas o asociadas a nodos de alta demanda.
- 4) Usar algoritmo genético para activar/desactivar combinaciones de rutas críticas.
- 5) Para cada cromosoma generado por el algoritmo genético, resolver el modelo de flujo continuo.
- 6) Usar gradiente o refinamiento local para ajustar parámetros continuos secundarios.

10.2. Algoritmo genético para decisiones de alto impacto

Un cromosoma puede representar qué rutas críticas están activas:

$$\mathbf{y} = [y_1, y_2, \dots, y_L], \quad y_\ell \in \{0, 1\}. \quad (50)$$

Ejemplo:

$$\mathbf{y} = [1, 0, 1, 1, 0] \quad (51)$$

Esto significa que las rutas 1, 3 y 4 están activas, mientras que las rutas 2 y 5 están cerradas o no seleccionadas.

La función de aptitud puede definirse como:

$$\text{fitness}(\mathbf{y}) = Z(\mathbf{y}) + \eta_1 \sum_{k \in D} w_k + \eta_2 \sum_{(i,j) \in E} \max\{0, x_{ij} - u_{ij}\}, \quad (52)$$

Donde $Z(\mathbf{y})$ es el costo óptimo obtenido al resolver el modelo de flujo con las rutas activas indicadas por $\mathbf{y}$.

Listing 3: Esquema del algoritmo genético integrado con el modelo lineal

```

Entrada: red logística, conjunto de rutas críticas, población inicial
Crear población de cromosomas y
Para generacion = 1 hasta G:
    Para cada cromosoma y:
        Activar/desactivar rutas según y
        Resolver el modelo lineal de flujo
        Calcular fitness = costo + penalizaciones
    Seleccionar mejores cromosomas
    Cruzar cromosomas seleccionados
    Mutar algunas posiciones
    Crear nueva población
Salida: mejor cromosoma y mejor solución de flujo asociada

```

10.3. Gradiente para refinamiento local

El método de gradiente es útil cuando se tiene una función continua y derivable. En este proyecto puede usarse para ajustar parámetros como ponderaciones de costo, penalizaciones o redistribuciones pequeñas de flujo alrededor de una solución factible.

Una forma general de actualización es:

$$\boldsymbol{\theta}^{(r+1)} = \boldsymbol{\theta}^{(r)} - \gamma \nabla J(\boldsymbol{\theta}^{(r)}), \quad (53)$$

Donde:

- $\boldsymbol{\theta}$ representa parámetros continuos ajustables;

- J es una función de costo diferenciable;
- γ es la tasa de aprendizaje;
- r es la iteración.

Para no mezclar conceptos incorrectamente, se recomienda explicar el gradiente como **refinamiento local**, no como sustituto del modelo de programación lineal.

11. Análisis de sensibilidad

El análisis de sensibilidad permite responder preguntas de tipo *what-if*. En la aplicación, el usuario podrá cambiar variables y observar cómo se modifica la solución.

11.1. Escenario 1: aumento del combustible

Si el combustible aumenta 15 % en rutas del Meta, se modifica el costo de esas rutas:

$$c_{ij}^{\text{nuevo}} = 1.15c_{ij}, \quad \forall (i, j) \in E_{\text{Meta}}. \quad (54)$$

Luego se vuelve a resolver el modelo y se compara:

$$\Delta Z = Z_{\text{nuevo}} - Z_{\text{base}}. \quad (55)$$

11.2. Escenario 2: cierre de vía principal

Si una vía se cierra, se elimina la arista o se fuerza su flujo a cero:

$$y_{ij} = 0 \quad \text{o equivalentemente} \quad x_{ij} = 0. \quad (56)$$

El modelo debe buscar rutas alternativas. Si no existen rutas suficientes, aparecerá demanda incumplida w_k .

11.3. Escenario 3: falla masiva de calidad

Si un acopio falla en calidad:

$$\bar{q}_j = 0. \quad (57)$$

Entonces:

$$x_{jk} \leq u_{jk}\bar{q}_j = 0. \quad (58)$$

Esto obliga al modelo a desviar el flujo hacia otros acopios o a registrar demanda incumplida si la red no tiene capacidad suficiente.

12. Ejemplo aplicado de validación del modelo

Esta sección presenta un caso pequeño para comprobar que el modelo funciona. Los datos son didácticos y escalados. No representan valores reales de la empresa, pero sí reproducen la lógica del problema: oferta, demanda, acopios, costos, capacidades y merma.

12.1. Estructura del caso

Se consideran:

- 3 estaciones de producción: O_1, O_2, O_3 .
- 2 centros de acopio: A_1, A_2 .
- 5 supermercados: D_1, D_2, D_3, D_4, D_5 .

12.2. Datos de oferta, capacidad y merma

Cuadro 7: Oferta de estaciones de producción

Origen	Nombre didáctico	Oferta (ton)
O_1	Estación Meta 1	90
O_2	Estación Meta 2	80
O_3	Estación Cundinamarca 1	100

Cuadro 8: Centros de acopio

Acopio	Nombre didáctico	Capacidad H_j (ton)	Merma ρ_j
A_1	Villavicencio	145	0.02
A_2	Bogotá	135	0.03

Cuadro 9: Demanda de supermercados

Destino	D_1	D_2	D_3	D_4	D_5
Demanda (ton)	45	40	55	35	50

La demanda total es:

$$45 + 40 + 55 + 35 + 50 = 225 \text{ toneladas.} \quad (59)$$

La oferta total es:

$$90 + 80 + 100 = 270 \text{ toneladas.} \quad (60)$$

La oferta es mayor que la demanda porque se necesita compensar la merma y porque no necesariamente se debe usar toda la producción disponible.

12.3. Costos unitarios de transporte

Cuadro 10: Costos desde orígenes hacia acopios

Ruta	Costo unitario	Capacidad de ruta (ton)
$O_1 \rightarrow A_1$	5	85
$O_1 \rightarrow A_2$	8	60
$O_2 \rightarrow A_1$	6	70
$O_2 \rightarrow A_2$	7	65
$O_3 \rightarrow A_1$	9	60
$O_3 \rightarrow A_2$	4	95

Cuadro 11: Costos desde acopios hacia supermercados

Acopio	D_1	D_2	D_3	D_4	D_5
A_1	4	6	7	10	9
A_2	9	7	5	4	6

12.4. Modelo específico del ejemplo

Para este ejemplo se minimiza solamente el costo de transporte, incorporando merma en el balance de acopios:

$$\min Z = \sum_{i \in O} \sum_{j \in A} c_{ij} x_{ij} + \sum_{j \in A} \sum_{k \in D} c_{jk} x_{jk}. \quad (61)$$

Sujeto a:

$$\sum_{j \in A} x_{ij} \leq b_i, \quad \forall i \in O, \quad (62)$$

$$\sum_{i \in O} x_{ij} \leq H_j, \quad \forall j \in A, \quad (63)$$

$$\sum_{k \in D} x_{jk} = (1 - \rho_j) \sum_{i \in O} x_{ij}, \quad \forall j \in A, \quad (64)$$

$$\sum_{j \in A} x_{jk} = d_k, \quad \forall k \in D, \quad (65)$$

$$0 \leq x_{ij} \leq u_{ij}, \quad \forall (i, j) \in E. \quad (66)$$

En este caso no se permite incumplimiento de demanda, por eso no se usa la variable w_k .

12.5. Solución óptima obtenida

La solución óptima del caso pequeño es la siguiente.

Cuadro 12: Flujos óptimos desde orígenes hacia acopios

Ruta	Flujo óptimo (ton)	Costo unitario	Costo total
$O_1 \rightarrow A_1$	85.0000	5	425.0000
$O_1 \rightarrow A_2$	0.0000	8	0.0000
$O_2 \rightarrow A_1$	10.9694	6	65.8164
$O_2 \rightarrow A_2$	40.0000	7	280.0000
$O_3 \rightarrow A_1$	0.0000	9	0.0000
$O_3 \rightarrow A_2$	95.0000	4	380.0000
Total	230.9694	–	1150.8164

Cuadro 13: Flujos óptimos desde acopios hacia supermercados

Ruta	Flujo óptimo (ton)	Costo unitario	Costo total
$A_1 \rightarrow D_1$	45.0000	4	180.0000
$A_1 \rightarrow D_2$	40.0000	6	240.0000
$A_1 \rightarrow D_3$	9.0500	7	63.3500
$A_1 \rightarrow D_4$	0.0000	10	0.0000
$A_1 \rightarrow D_5$	0.0000	9	0.0000
$A_2 \rightarrow D_1$	0.0000	9	0.0000
$A_2 \rightarrow D_2$	0.0000	7	0.0000
$A_2 \rightarrow D_3$	45.9500	5	229.7500
$A_2 \rightarrow D_4$	35.0000	4	140.0000
$A_2 \rightarrow D_5$	50.0000	6	300.0000
Total	225.0000	–	1153.1000

El costo total mínimo es:

$$Z^* = 1150.8164 + 1153.1000 \quad (67)$$

$$= 2303.9164. \quad (68)$$

Si el precio de venta promedio fuera $p = 18$ unidades monetarias por tonelada, el ingreso por demanda atendida sería:

$$I = 18(225) = 4050. \quad (69)$$

La ganancia proyectada simplificada sería:

$$U = 4050 - 2303.9164 = 1746.0836. \quad (70)$$

12.6. Verificación de restricciones

12.6.1. Verificación de oferta

$$O_1 : \quad 85.0000 + 0.0000 = 85.0000 \leq 90, \quad (71)$$

$$O_2 : \quad 10.9694 + 40.0000 = 50.9694 \leq 80, \quad (72)$$

$$O_3 : \quad 0.0000 + 95.0000 = 95.0000 \leq 100. \quad (73)$$

Ningún origen envía más de lo que tiene disponible.

12.6.2. Verificación de capacidad de acopios

Entrada a A_1 :

$$85.0000 + 10.9694 + 0.0000 = 95.9694 \leq 145. \quad (74)$$

Entrada a A_2 :

$$0.0000 + 40.0000 + 95.0000 = 135.0000 \leq 135. \quad (75)$$

El acopio A_2 trabaja exactamente al límite de su capacidad. Por tanto, A_2 es un nodo crítico del sistema.

12.6.3. Verificación de balance con merma

Para A_1 , la entrada es 95.9694 toneladas y la merma es 2 %:

$$(1 - 0.02)(95.9694) = 0.98(95.9694) = 94.0500. \quad (76)$$

La salida desde A_1 es:

$$45.0000 + 40.0000 + 9.0500 + 0 + 0 = 94.0500. \quad (77)$$

Por tanto, el balance se cumple.

Para A_2 , la entrada es 135.0000 toneladas y la merma es 3 %:

$$(1 - 0.03)(135.0000) = 0.97(135.0000) = 130.9500. \quad (78)$$

La salida desde A_2 es:

$$0 + 0 + 45.9500 + 35.0000 + 50.0000 = 130.9500. \quad (79)$$

Por tanto, el balance también se cumple.

12.6.4. Verificación de demanda

$$D_1 : \quad 45.0000 + 0.0000 = 45, \quad (80)$$

$$D_2 : \quad 40.0000 + 0.0000 = 40, \quad (81)$$

$$D_3 : \quad 9.0500 + 45.9500 = 55, \quad (82)$$

$$D_4 : \quad 0.0000 + 35.0000 = 35, \quad (83)$$

$$D_5 : \quad 0.0000 + 50.0000 = 50. \quad (84)$$

Todos los supermercados reciben exactamente la demanda solicitada.

12.6.5. Verificación de rutas saturadas

Al comparar flujo con capacidad, se identifican rutas críticas:

- $O_1 \rightarrow A_1$: flujo 85 sobre capacidad 85; uso 100 %.

- $O_3 \rightarrow A_2$: flujo 95 sobre capacidad 95; uso 100 %.
- Acopio A_2 : entrada 135 sobre capacidad 135; uso 100 %.

Estas rutas y nodos deben resaltarse visualmente en la aplicación porque cualquier aumento de demanda o cierre de vía podría afectar la factibilidad de la red.

12.7. Interpretación logística de la solución

La solución tiene sentido desde el punto de vista operativo:

- El modelo utiliza fuertemente $O_1 \rightarrow A_1$ porque es una ruta barata hacia el acopio que atiende mejor a D_1 y D_2 .
- El modelo utiliza $O_3 \rightarrow A_2$ porque es la ruta más barata hacia el acopio que atiende mejor a D_3 , D_4 y D_5 .
- El supermercado D_3 se divide entre ambos acopios porque la red necesita balancear capacidad y merma.
- El acopio A_2 queda saturado, lo que indica un cuello de botella importante.

Esto demuestra que el modelo no escoge rutas de manera aislada. Toma decisiones considerando simultáneamente costos, capacidades, demanda y pérdidas por merma.

13. Mini análisis de sensibilidad sobre el caso aplicado

Aunque el análisis formal de sensibilidad se desarrolla en la fase de resultados, aquí se muestra una comprobación breve para evidenciar que el modelo responde a cambios en las condiciones.

13.1. Aumento del 15 % en costos de rutas del Meta

Supóngase que las rutas asociadas a O_1 y O_2 aumentan su costo en 15 %. Al resolver nuevamente el modelo, se obtiene:

$$Z_{\text{combustible}} = 2419.5388. \quad (85)$$

La diferencia frente al caso base es:

$$\Delta Z = 2419.5388 - 2303.9164 = 115.6224. \quad (86)$$

El incremento porcentual del costo total es:

$$\frac{115.6224}{2303.9164} \times 100 = 5.02 \%. \quad (87)$$

Esto significa que un aumento del 15 % en ciertas rutas no necesariamente aumenta el costo total en 15 %, porque solo una parte del costo total depende de esas rutas. Esta es precisamente la utilidad del análisis de sensibilidad.

13.2. Cierre de la ruta $O_1 \rightarrow A_1$

Si la ruta $O_1 \rightarrow A_1$ se cierra, se fuerza:

$$x_{O_1 A_1} = 0. \quad (88)$$

Al resolver nuevamente, el costo óptimo cambia a:

$$Z_{\text{cierre}} = 2580.7020. \quad (89)$$

El aumento frente al caso base es:

$$\Delta Z = 2580.7020 - 2303.9164 = 276.7856. \quad (90)$$

El incremento porcentual es:

$$\frac{276.7856}{2303.9164} \times 100 = 12.01 \%. \quad (91)$$

Esto muestra que la ruta $O_1 \rightarrow A_1$ es crítica. Aunque el modelo todavía puede cumplir la demanda, debe redirigir producto por rutas más costosas.

14. Traducción del modelo a Python

En Python, el modelo puede implementarse con matrices. La idea general es:

- 1) Crear un vector de variables $\mathbf{x}$ con todos los flujos.
- 2) Crear un vector de costos $\mathbf{c}$.
- 3) Construir matrices de restricciones de desigualdad $A_{ub}\mathbf{x} \leq \mathbf{b}_{ub}$.
- 4) Construir matrices de restricciones de igualdad $A_{eq}\mathbf{x} = \mathbf{b}_{eq}$.
- 5) Resolver el problema de minimización.
- 6) Convertir el vector solución en tablas de rutas y flujos.

La estructura computacional sería:

Listing 4: Estructura sugerida de implementación en Python

```
1. Cargar nodos.csv, rutas.csv, demanda.csv y calidad.csv
2. Construir conjuntos O, A, D y E
3. Crear vector de costos c
4. Crear bounds: 0 <= x_ij <= capacidad_ij
5. Crear restricciones de oferta
6. Crear restricciones de capacidad de acopios
7. Crear restricciones de balance con merma
8. Crear restricciones de demanda
9. Resolver el modelo lineal
10. Calcular costo total, ganancia, rutas críticas y cuellos de botella
11. Mostrar resultados en tablas y grafo
```

15. Conclusiones de la fase matemática

El modelo presentado permite representar formalmente la red logística de Acuícola Real del Meta como un problema de optimización sobre grafos. La formulación considera los elementos exigidos por el problema: oferta, demanda, capacidad de rutas, capacidad de acopios, equilibrio de flujo, merma, calidad, rutas óptimas y sensibilidad ante escenarios.

El ejemplo aplicado demuestra que el modelo produce una solución coherente y verificable. Se cumplen las restricciones de oferta, demanda, capacidad y balance con merma. Además, el

análisis permite identificar rutas y nodos críticos, como la saturación del acopio A_2 y de las rutas $O_1 \rightarrow A_1$ y $O_3 \rightarrow A_2$.

Para el desarrollo del programa en Python, esta formulación debe ser la base del motor de cálculo. La interfaz gráfica, los mapas y los escenarios what-if deben construirse encima de este modelo, no al revés. De esta forma, el proyecto mantiene rigor matemático y coherencia computacional.

Referencias bibliográficas sugeridas

Referencias

- [1] Taha, H. A. *Investigación de operaciones*. Pearson Educación.
- [2] Hillier, F. S. y Lieberman, G. J. *Introducción a la investigación de operaciones*. McGraw-Hill.
- [3] Ahuja, R. K., Magnanti, T. L. y Orlin, J. B. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
- [4] Nocedal, J. y Wright, S. J. *Numerical Optimization*. Springer.
- [5] Goldberg, D. E. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.
- [6] Bazaraa, M. S., Jarvis, J. J. y Sherali, H. D. *Linear Programming and Network Flows*. Wiley.

A. Resumen de símbolos principales

Símbolo	Significado
$G = (V, E)$	Grafo dirigido ponderado de la red logística.
O	Conjunto de estaciones de producción.
A	Conjunto de centros de acopio.
D	Conjunto de supermercados.
x_{ij}	Toneladas transportadas desde i hacia j .
s_j	Stock final en el acopio j .
m_j	Merma o pérdida en el acopio j .

w_k	Demanda incumplida en el supermercado k .
y_{ij}	Variable binaria que indica si la ruta (i, j) está activa.
b_i	Oferta disponible en el origen i .
d_k	Demanda del supermercado k .
c_{ij}	Costo unitario de transportar por la ruta (i, j) .
u_{ij}	Capacidad máxima de la ruta (i, j) .
H_j	Capacidad del acopio j .
ρ_j	Tasa de merma en el acopio j .
$\bar{q}_j$	Estado de calidad del acopio j .
Z	Costo total a minimizar.
U	Ganancia proyectada.

B. Recomendación para defensa oral

Para explicar este modelo ante el profesor, se recomienda seguir este orden:

- 1) Primero explicar la red: estaciones, acopios y supermercados.
- 2) Luego explicar que cada ruta tiene costo y capacidad.
- 3) Después presentar la variable principal x_{ij} como toneladas enviadas.
- 4) Explicar la función objetivo: minimizar todos los costos.
- 5) Explicar las restricciones una por una con lenguaje sencillo.
- 6) Mostrar el ejemplo aplicado y verificar oferta, demanda y balance.
- 7) Concluir con los escenarios de sensibilidad y la integración futura con Python.

La frase central de la sustentación puede ser:

El modelo decide cuántas toneladas enviar por cada ruta de la red, minimizando el costo total y respetando oferta, demanda, capacidad, merma y calidad. La teoría de grafos permite representar la red, mientras que la programación lineal calcula la asignación óptima de flujo.